The main section of the program performs two functions. In lines 6 to 13, the data-address-generators (DAGs) are configured to manage the circular buffers: dline[ ], and coef[ ]. As described in the last chapter, three parameters are needed for each buffer: the starting location of the buffer in memory (b0 and b8), the length of the buffer (l0 and l8), and the step size of the data being stored in the buffer (m0 and m8). These parameters that control the circular buffers are stored in hardware registers in the DAGs, allowing them to access and manage the data very efficiently.

The second action of the main program is a "thumb-twiddling" loop, implemented in lines 15 to 19. This does nothing but wait for an interrupt indicating that an input sample has been acquired. All of the processing in this program occurs on a **sample-by-sample** basis. Each time a sample is read from the input, a sample in the output signal is calculated and routed to the codec. Most time-domain algorithms, such as FIR and IIR filters, fall into this category. The alternative is **frame-by-frame** processing, which is required for frequency-domain techniques. In the frame-by-frame method, a *group* of samples is read from the input, calculations are conducted, and a *group* of samples is written to the output.

The subroutine that services the sample-ready interrupt is broken into three sections. The first section (lines 27 to 33) fetches the sample from the codec as a fixed point number, and converts it to floating point. In SHARC assembly language, a data register holding a fixed point number is referred to by "r" (such as r0, r8, r15, etc.), and by "f" if it is holding a floating point number (i.e., f0, f8, or f15.). For instance, in line 32, the fixed point number in data register 0 (i.e., r0) is converted into a floating point number and overwrites data register 0 (i.e., f0). This conversion is done according to a scaling specified by the fixed point number in data register 1 (i.e. r1). In the third section (lines 47 to 53), the opposite steps take place; the floating point number for the output sample is converted to fixed point and sent to the codec.

The FIR filter that converts the input samples into the output samples is contained in lines 35 to 45. All the calculations are carried out in floating point, avoiding the need to worry about scaling and overflow. As described in the last chapter, this section of code is optimized to take advantage of the SHARC DSP's ability to execute multiple instructions each clock cycle.

After we have the assembly program written and the filter kernel designed, we are ready to create a program that can be executed on the SHARC DSP. This is done by running the *compiler*, the *assembler*, and then the *linker;* three programs provided with the EZ-KIT Lite. The compiler converts a C program into the SHARC's assembly language. If you directly write the program in assembly, such as in this example, you bypass this step. The assembler and linker convert the program and external files (such as the architecture file, codec initialization routines, filter kernel, etc.) into the final executable file. All this takes about 30 seconds, with the final result being a SHARC program residing on the harddisk of your PC. The EZ-KIT Lite host is then used to run the program on the EZ-KIT Lite. Simply click